# Project Specification: Agent-Based Economic Simulator (Phase 1)

## Overview

This project is Phase 1 of an Agent-Based Model (ABM) designed to simulate a localized, natural-selection-based capitalist economy. The long-term goal of the project is to create a baseline simulation of "survival of the fittest" economics, which will later be compared against a centrally optimized "gradient descent" economic model.

For this phase, the agent must build a procedural world where populations gather resources, invest in distinct strategic traits, expand, and compete.

## Tech Stack

- **Language:** Python 3.x
- **Simulation Engine:** `mesa` (Framework for Agent-Based Modeling)
- **Math & Terrain:** `numpy`
- **Data Collection:** `pandas` , `matplotlib` (for visual rendering)

## File Architecture

The project should be divided into three primary files:

1. `agents.py` - Contains the logic for the interactive populations and the static resource tiles.
2. `model.py` - Contains the `mesa.Model` class, handling world generation, grid management, and turn scheduling.
3. `run.py` - The execution script to initialize the model, run the loop, and extract data.

## Class Blueprints

### 1. `agents.py`

**Class:** `ResourceCell(mesa.Agent)`

Acts as the static environmental layer.

- **Attributes:**
  - `terrain_type` : Enum or string ("Land" or "Water").
  - `resource_amount` : Current abstract resources available.

    - `max_capacity` : Maximum resources the cell can hold.
- **Visualization:** * "Land" should render as Green.
    - "Water" should render as Blue.
- **Behavior ( `step` ):**
    - If "Land", `resource_amount` regenerates by a fixed amount per tick, capped at `max_capacity` .

**Class:** `Population(mesa.Agent)`

The active competitors in the simulation. A single agent represents all inhabitants of a single tile.

- **Attributes:**
    - `inhabitant_count` : Integer representing the total number of people on this tile.
    - `stockpile` : Current accumulated resources.
    - `tech_level` : Integer representing current technological tier.
    - `lineage_color` : A unique color assigned at the simulation start to denote the "nation" or origin seed.
- **Investment Traits (Must always sum to 100%):** *These traits are shared by all inhabitants of the specific tile.*
    - `military_pct` : Allocation for combat/expansion.
    - `economic_pct` : Allocation for trade and resource efficiency.
    - `diplomatic_pct` : Allocation for treaties and trade leverage.
    - `tech_pct` : Allocation for technological advancement.
- **Behavior ( `step` ):**
    1. **Harvest:** Gather resources from the underlying `ResourceCell` and immediately adjacent cells.
    2. **Allocate:** Distribute harvested resources into internal buckets based on Investment Traits.
    3. **Advance Tech:** If the accumulated tech resources pass a threshold, increment `tech_level` .
    4. **Trait Drift (Generational Shift):** Over time (e.g., a tiny random percent shift each tick, or upon reaching certain population milestones), slightly alter the Investment Traits to simulate old generations dying off and new generations forming new beliefs. (Ensure traits always re-normalize to 100%).

5. **Expand/Migrate:** If `inhabitant_count` exceeds a threshold, a portion of the population travels to an adjacent tile. If empty, it spawns a new `Population` agent there, carrying over the parent's `lineage_color` and current Investment Traits.
6. **Diffusion:** Gain passive tech progress if bordering a population with a higher `tech_level`.

**2.** `model.py`

**Class:** `WorldModel(mesa.Model)`

- **Setup (`__init__`):**
  - Initialize `mesa.space.MultiGrid` (allowing Populations to sit on ResourceCells).
  - Initialize `mesa.time.RandomActivation` scheduler.
  - Generate a procedural map (e.g., using basic noise or clustering to create continents/oceans).
  - Populate the grid with `ResourceCell` agents.
  - Randomly place `N` starting `Population` agents on land cells. Assign each a unique `lineage_color` and randomized Investment Traits.
- **Behavior (`step`):**
  - Trigger the scheduler to step all agents.
  - Collect data via `mesa.DataCollector` (track max tech, surviving lineages, dominant traits).

## Core Mechanics & Mathematics

The coding agent should implement the following specific logic for interactions:

### 1. The Technology Multiplier

Technology does not produce goods directly; it is a multiplier.

- **Formula:** `Effective_Output = Base_Trait_Allocation * (1 + (tech_level * 0.05))`
- *Implementation:* When calculating combat strength or harvest efficiency, multiply the raw economic/military points by this modifier.

### 2. Economic Leverage (Trade Arbitrage)

Populations adjacent to each other can trade. Diplomacy bridges value gaps.

- **Logic:** Population A proposes a trade to B. B evaluates the resource values.

- **Diplomatic Modifier:** Calculate `Diplomatic_Delta = Population A Diplomatic Score - Population B Diplomatic Score`.
- If `Diplomatic_Delta > 0`, A's offer is perceived by B as more valuable.
- **Constraint:** The diplomatic bonus cannot exceed a 20% increase in the base perceived value of the goods. If the trade deficit is higher than 20%, B rejects it.

### 3. Political Leverage (Treaties & Expansion)

Populations expanding into occupied territory trigger competition.

- **Logic:** Before A attacks/absorbs B's tile, B attempts a Non-Aggression Pact (NAP).
- A calculates a `Land_Desire_Score` based on its own population pressure.
- B calculates a `Deterrence_Score` based heavily on its Diplomatic Output and partially on its Military Output.
- If `Deterrence_Score > Land_Desire_Score`, A cancels the expansion into B's tile. Otherwise, standard combat resolution occurs (comparing Effective Military outputs).

## Immediate Task for Coding Agent

1. Scaffold the three files ( `agents.py`, `model.py`, `run.py` ).
2. Implement the `WorldModel` with a basic procedural land/water grid and spawn the `ResourceCell` agents.
3. Render a Matplotlib grid of the initialized map. Verify that water is Blue, land is Green, and